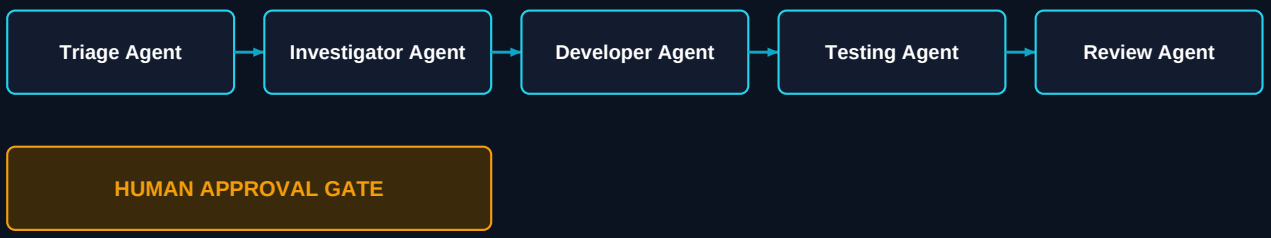
HACKATHON PROJECT DOCUMENTATION | VERSION 1.0

RELAY AI

The Autonomous AI Engineering Team.

A multi-agent workspace that turns a reported software bug into an investigated, tested and human-approved code change.



Proposed solution and implementation blueprint for a four-member team
Prepared: 04 November 2026
Target events: Diwali Codefest 2026 (Build for Builders);
Flip the Switch Hackathon (Best Business Workflow, verify official track rules)

Status legend: all capabilities are PROPOSED unless explicitly marked IMPLEMENTED. No performance claims are made.

Contents

Part A. Product and Business Understanding . . . 3
2. Executive summary . . . 3
3. Project introduction . . . 3
4. Problem statement . . . 3
5. Existing workflow and pain points . . . 3
6. Proposed solution . . . 4
7. Product vision and mission . . . 4
8. Objectives . . . 4
9. Scope and limitations . . . 4
10. Target customers and user personas . . . 4
11. Business value proposition . . . 5
12. Use cases . . . 5
13. Competitive positioning and differentiation . . . 5

Part B. System and Technical Design . . . 6
14. Functional requirements . . . 6
15. Non-functional requirements . . . 6
16. System architecture . . . 6
17. Architecture component descriptions . . . 7
18. Multi-agent workflow . . . 7
19. Detailed responsibilities of every AI agent . . . 8
20. Agent communication and structured handoffs . . . 8
21. Technology stack with justification . . . 9
22. Frontend architecture . . . 9
23. Backend architecture . . . 9
24. Database design and proposed entities . . . 9
25. GitHub integration . . . 9
26. Switch integration and shared context . . . 10
27. Docker execution and sandbox design . . . 10
28. Human approval mechanism . . . 10
29. Security, privacy and permissions . . . 10
30. Error handling and recovery . . . 11
31. API design (proposed) . . . 11

Part C. Implementation and Validation . . . 12
32. Four-member team responsibilities . . . 12
33. MVP features . . . 12
34. Prioritised feature roadmap . . . 12
35. Development methodology . . . 12
36. Implementation phases . . . 12
37. Testing strategy . . . 13
38. Functional and integration test cases . . . 13
39. Performance and reliability metrics . . . 13
40. Risks and mitigation . . . 13
41. Known limitations . . . 14
42. Future enhancements . . . 14
43. Deployment considerations . . . 14
44. Conclusion . . . 14
Appendix A. Assumptions requiring verification . . . 14
Appendix B. Example issue and resolution report (illustrative) . . . 14
Appendix C. Suggested repository structure . . . 15

Part A. Product and Business Understanding

2. Executive summary

RELAY AI is a multi-agent engineering workspace that turns a software bug reported in a team chat into a structured, investigated, tested and human-approved code change. Specialist AI agents classify the issue, inspect a connected GitHub repository, propose a patch in an isolated workspace, run automated tests in a Docker sandbox and review the result. A human developer then approves or rejects the change before any pull request is created.

The product is built on one distinction: agents must do verifiable work, not merely exchange messages. Every stage produces an inspectable artifact (task record, repository evidence, diff, test report, review findings) and passes structured output to the next stage under the control of an Orchestrator.

Status. This document is a proposed design and build plan for a four-member student team. Technologies named are a proposed stack, not implemented features. Competition rules, dates, tracks and the Switch integration method must be verified against official sources (see Appendix A).

3. Project introduction

RELAY AI is designed for two hackathon entries from one core workflow: Diwali Codefest 2026 (proposed track: Build for Builders, emphasising impact, implementation, originality, usability and presentation) and the Flip the Switch Hackathon (proposed track: Best Business Workflow, emphasising genuine Switch integration). The demo for each event is tailored, while the product, terminology and architecture are identical.

Item	Detail
Project	RELAY AI
Tagline	The Autonomous AI Engineering Team
Category	Multi-agent AI workflow / developer productivity
Team	Four members (see section 32)
Core output	Task record, evidence, diff, test report, review, human decision, pull request

4. Problem statement

In small software teams a bug report triggers a fragmented sequence: someone clarifies the issue, another person searches the repository, a developer attempts a fix, tests are run in a terminal and a reviewer checks the change. Work is spread across chat messages, code hosting, terminals and individual memory. Context is lost, responsibilities are unclear and progress is hard to audit. AI coding assistants help with individual steps, but their suggestions often arrive without test evidence, shared team context or a clear approval boundary.

5. Existing workflow and pain points

Stage	Typical current practice	Pain point
Report	Free-form chat message	Missing details; no structured task
Investigation	Manual repository search	Repeated effort; findings stay in memory
Fix	Developer writes patch locally	Rationale rarely recorded with the change
Verification	Tests run in a personal terminal	Output pasted or summarised, not auditable
Review	Separate review thread	Evidence and context must be reassembled
Status	Ask in chat	No single view of progress or owner

6. Proposed solution

RELAY AI connects a Switch-connected team room, a GitHub repository and a group of specialist agents through one Orchestrator. A teammate posts a bug; the system creates a trackable task and drives it through triage, investigation, patch proposal, testing and review. At the human approval gate, a developer inspects the diff and evidence and decides. Only after approval is a pull request created and a resolution report posted to the same room. No automatic merge or production deployment is performed.

7. Product vision and mission

Vision. An AI engineering team that is a reliable collaborator inside the team's existing workflow, with clear responsibility, evidence and human control.

Mission. Reduce the coordination overhead between issue reporting, investigation, implementation and verification, while keeping humans in control of every code change.

8. Objectives

- Accept a bug report from a supported team chat and convert it into a structured task.
- Inspect a connected repository and cite files or code locations as evidence.
- Propose a minimal fix in an isolated workspace and run relevant tests in a restricted container.
- Review the change and request explicit human approval before creating a pull request.
- Keep a timeline of agent actions, artifacts, failures and decisions.
- Deliver one reliable, repeatable end-to-end demo rather than many unfinished features.

9. Scope and limitations

In scope (hackathon MVP)	Out of scope
One Switch-connected team room and one supported chat provider	Autonomous production deployment
One GitHub demo repository	Unrestricted access to arbitrary repositories or credentials
One language or a deliberately narrow project type	Every language and build system
Five specialist agents under a deterministic workflow	Multiple chat platforms at once
One seeded, reproducible bug scenario	Voice commands, analytics, broad project management
Human approval before any externally visible change	Automatic merge to a protected branch

Limitations: AI-generated patches are not guaranteed correct; a passing test suite does not prove absence of regressions; results from one seeded bug do not generalise to arbitrary defects.

10. Target customers and user personas

Persona	Context	Need	RELAY AI value
Startup engineer	Few engineers, constant context switching	Faster, traceable bug handling	One view of status, evidence and diffs
Student / hackathon team	Parallel work, limited review capacity	Structured triage, tested proposals	Repeatable workflow, clear ownership
Small-repo maintainer	Incoming reports need clarifying	Clear tasks with file leads	Structured task and investigation
Technical lead	Needs visibility and control	Audit trail and approval authority	Timeline, evidence, approval gate

11. Business value proposition

RELAY AI aims to reduce handoff overhead, preserve investigation context, make AI contributions verifiable and keep approval authority with humans. These are design goals. Value will be measured during rehearsals (time from report to review-ready patch, share of proposed patches passing the selected tests, human accept/reject rate, unsupported claims per report). No market size, customer, revenue or performance claims are made in this document.

12. Use cases

ID	Use case	Outcome
UC-1	Report a clear bug in the team room	Task created; investigation starts
UC-2	Report an incomplete bug	Triage Agent requests missing details; task NEEDS_INFORMATION
UC-3	Review a proposed patch	Human sees diff, test output and review findings; approves or rejects
UC-4	Reject a patch	No pull request or merge occurs; reason recorded
UC-5	Audit a past task	Timeline shows agent, input, output and decision

13. Competitive positioning and differentiation

General workflow categories (not claims about specific products): chat-based coordination, issue trackers, standalone AI coding assistants and CI pipelines each cover part of the flow. RELAY AI positions itself as the orchestration layer across the whole bug-to-fix path.

Dimension	Typical chat-only assistant	RELAY AI (design goal)
Task record	Not by default	Structured record per issue
Stage artifacts	Single response	Evidence, diff, tests, review
Test evidence	Varies	Command, exit code, logs recorded
Approval	Varies	Mandatory human gate
Team context	Often a separate tool	Shared Switch-connected room

This is an illustrative comparison of design goals, not a benchmark.

Part B. System and Technical Design

14. Functional requirements

ID	Requirement	Acceptance evidence	Pri.
FR-01	Receive an issue from the configured chat room	Message creates a task with unique ID	P0
FR-02	Normalise issue into structured fields	Title, severity, repository, criteria visible	P0
FR-03	Investigate the connected repository	Relevant files and evidence-backed findings	P0
FR-04	Generate a proposed code change	Diff produced in isolated workspace	P0
FR-05	Run relevant tests	Command, exit status, output recorded	P0
FR-06	Review the proposed change	Findings, risks, open questions	P0
FR-07	Require human approval	No PR or protected action before approval	P0
FR-08	Publish resolution summary	Chat receives status, links, remaining risks	P0
FR-09	Create GitHub pull request	PR created only after approval	P1
FR-10	Show task timeline and agent status	Stage, owner, timestamps, artifacts	P1
FR-11	Support multiple repos and platforms	Configurable integrations	P2

15. Non-functional requirements

Area	Requirement
Security	Least-privilege GitHub token; secrets never in prompts, logs or chat
Isolation	Repository commands and tests run in a disposable, restricted container
Human control	Explicit approval for PR creation and any merge or deploy
Traceability	Record agent, input reference, output, timestamps, action results
Reliability	Handle timeouts, malformed model output, tool errors and test failures without silently advancing
Transparency	Distinguish verified results from model hypotheses; show uncertainty
Performance	Show progress and intermediate status; no hard latency promise
Privacy	Process only authorised repositories and chat data

16. System architecture

The architecture separates the communication layer, orchestration layer, specialist agents, tool adapters and evidence store. The human approval gate sits between the agents and any externally visible action.

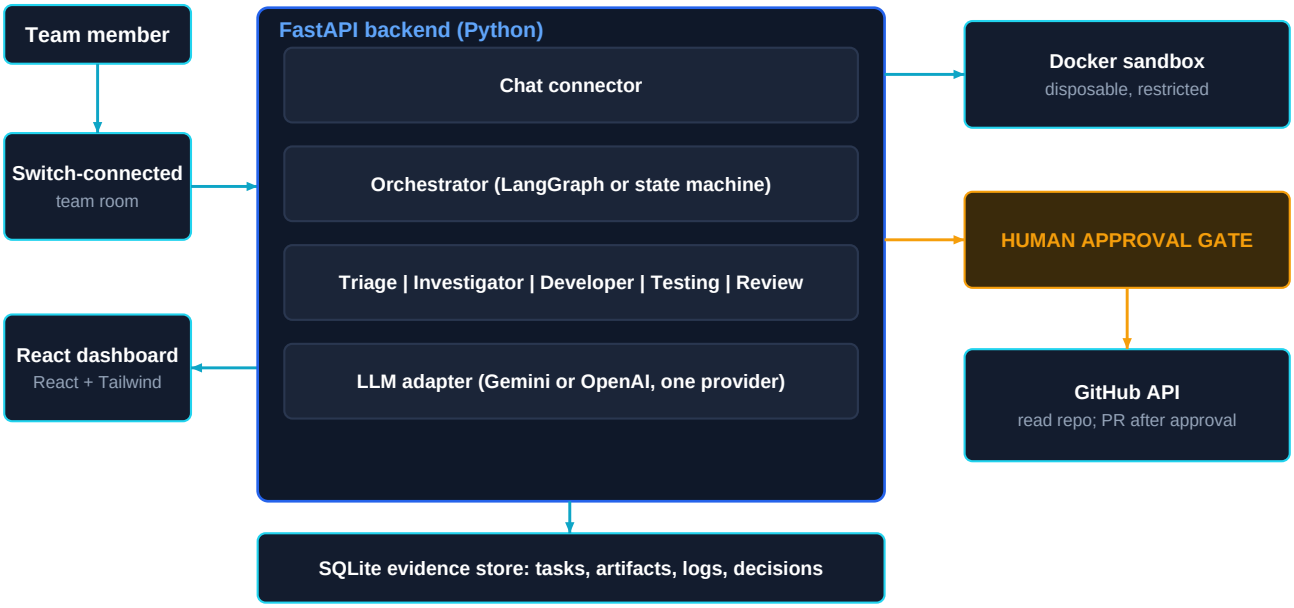


Figure 1. Proposed system architecture (proposed stack; human approval gate shown before GitHub pull request creation).

17. Architecture component descriptions

Component	Responsibility	Dependency
Chat connector	Receives messages, posts status through the supported chat / Switch interface	Switch setup per official docs
Orchestrator	Owns task state, sequence, schema validation, retries, audit events	None (own code)
Agent layer	Specialised tasks with constrained tools and output schemas	LLM API
Repository adapter	Reads permitted files, branches, metadata; creates PR after approval	GitHub API token
Execution sandbox	Runs approved test commands in a disposable container	Docker
Evidence store	Persists tasks, artifacts, logs, decisions	SQLite (MVP)
Dashboard	Shows status, agent activity, diff, test results, approval controls	Backend API

18. Multi-agent workflow

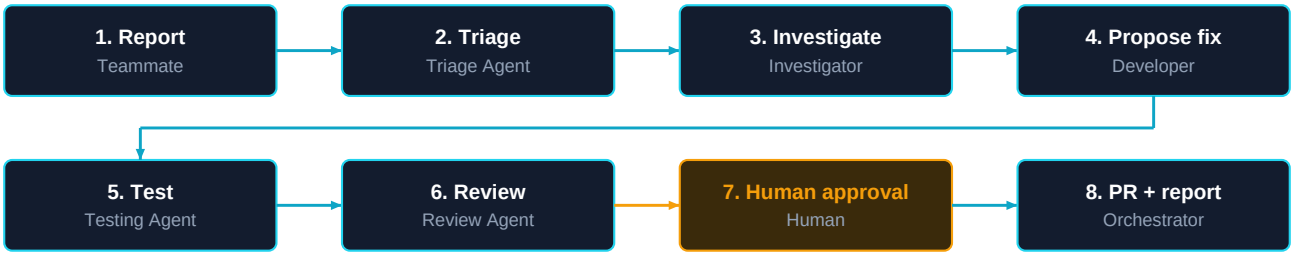


Figure 2. End-to-end workflow with the human approval gate (stage 7).

Workflow states: RECEIVED, TRIAGED, INVESTIGATING, FIX_PROPOSED, TESTING, REVIEW_READY, AWAITING_APPROVAL, APPROVED, PR_CREATED, RESOLVED. **Alternative terminal states:** NEEDS_INFORMATION, TEST_FAILED, REJECTED, BLOCKED, ERROR. The workflow is deterministic: the Orchestrator, not an agent, decides the next state.

19. Detailed responsibilities of every AI agent

Agent	Receives	Does	Returns
Triage Agent	Raw chat report	Classifies, sets severity, flags missing details, defines acceptance criteria	Structured issue object with confidence
Investigator Agent	Task, permitted repo files	Searches code, forms root-cause hypothesis, cites evidence	Findings with file paths, snippets, uncertainty
Developer Agent	Findings, isolated workspace	Prepares the smallest reasonable candidate fix	Diff, files changed, rationale, limitations
Testing Agent	Diff, test command allow-list	Runs selected tests in Docker sandbox	Commands, exit codes, logs, pass/fail
Review Agent	Diff, test evidence, criteria	Checks correctness, security, regression risk	Findings by severity, recommendation
Orchestrator	All agent outputs	Controls states, validates schemas, retries, audit	Timeline, status, events

20. Agent communication and structured handoffs

Each agent receives only the context and tools needed for its stage and returns structured data validated by the Orchestrator before the workflow advances. Free-form prose alone is never an acceptable output.

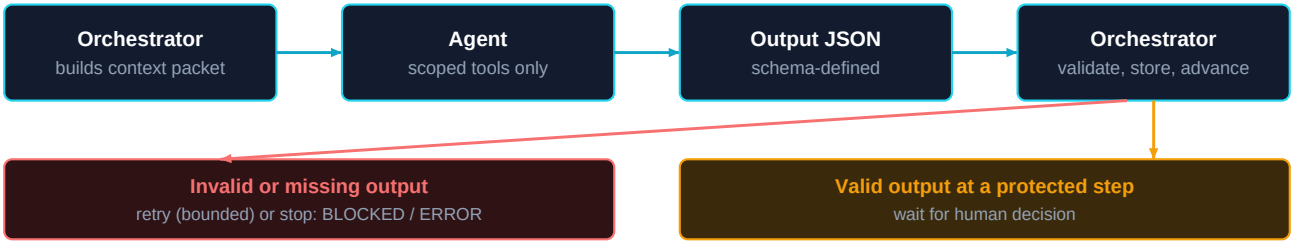


Figure 3. Structured handoff and failure paths.

```
{
  "task_id": "REL-001",
  "agent": "investigator",
  "status": "completed",
  "summary": "Likely discount calculation defect",
  "evidence": [{"file": "src/checkout.py", "location": "calculate_total",
                "reason": "Discount applied twice"}],
  "confidence": 0.82,
  "uncertainties": ["No integration test for stacked discounts"],
  "recommended_next_step": "developer"
}
```

The example is illustrative; the confidence value is a model self-report, not a measured probability, and is displayed as such.

21. Technology stack with justification

Layer	Proposed option	Justification
Frontend	React + Tailwind CSS	Fast dashboard development, reusable components
Backend	Python + FastAPI	Simple API layer, strong Python ecosystem, typed schemas
Orchestration	LangGraph or custom state machine	Use LangGraph if known; otherwise an explicit state machine is easier to debug
LLM	Gemini API or OpenAI API (choose one)	One provider behind an adapter limits integration risk
Collaboration	Switch + one supported chat provider	Central to the Flip the Switch entry; method per official docs
Repository	GitHub API	Read context, create PR after approval
Execution	Docker sandbox	Isolates tests and generated code
Database	SQLite	Zero-setup persistence for the MVP

Stack discipline. Do not integrate several LLM providers, frameworks or chat platforms at once. Finish one working path first.

22. Frontend architecture

- **Pages:** task list; task detail (timeline, evidence, diff, test report, review); approval panel.
- **State:** data fetched from the backend API; live status via polling for the MVP (WebSocket or server-sent events optional).
- **Components:** StatusTimeline, AgentCard, DiffViewer, TestReport, ApprovalButtons, TaskTable.
- **Rules:** approval buttons are enabled only in AWAITING_APPROVAL; the UI never claims tests passed unless the stored report says so.

23. Backend architecture

A FastAPI application exposes task and approval endpoints and receives chat events. Internal modules: `api/`, `orchestrator/`, `agents/`, `integrations/` (switch_chat, github), `sandbox/`, `schemas/` and `storage/`. Agents are plain functions behind a common interface; the LLM provider is wrapped in an adapter so it can be swapped or replaced by cached fixtures for the backup demo. Pydantic models define every handoff schema.

24. Database design and proposed entities

Entity	Key fields
tasks	task_id, reported_by, source_channel, repository, title, description, severity, acceptance_criteria, current_stage, final_status, created_at, updated_at
agent_runs	run_id, task_id, agent, input_ref, output_json, status, started_at, finished_at
artifacts	artifact_id, task_id, type (investigation, diff, test_report, review), content_ref
test_runs	test_id, task_id, command, exit_code, stdout_ref, duration, environment
approvals	approval_id, task_id, decision, decided_by, reason, decided_at
events	event_id, task_id, type, payload, created_at (audit timeline; chat event IDs for idempotency)

25. GitHub integration

- Use a fine-grained token scoped to the single demo repository: read contents, and write access to branches and pull requests only.
- Reading: list and fetch permitted files; clone into the sandbox workspace read-only where possible.
- Writing: push a feature branch and open a pull request only after an approval record exists. The protected branch is never modified.
- Dependency: GitHub API availability and token validity. On failure the workflow stops safely and reports the error (section 30).

26. Switch integration and shared context

Switch is intended to be the shared collaboration and context layer: the bug is reported in the team room, agents use shared context, and status and the final resolution report are posted back to the same room. The Chat connector isolates this dependency behind a small interface (receive event, post message).

To verify. The supported Switch SDK or API method, supported chat providers, event model and setup steps must be taken from the official Switch documentation for the event. This document intentionally names no Switch endpoints and assumes no capabilities beyond receiving a message and posting results. Handle duplicate events with idempotency keys.

27. Docker execution and sandbox design

Model-generated commands are never executed on a developer's host. Tests run in a disposable container created per task from a prebuilt image containing the demo repository dependencies. Only commands from a fixed allow-list (for example the demo test command) are accepted.

```
docker run --rm --network none --memory 512m --cpus 1 --pids-limit 128 \
  --read-only --tmpfs /tmp --user 1000:1000 \
  -v "$WORKSPACE":/work:rw -w /work relay-sandbox:demo \
  timeout 120 pytest tests/test_checkout.py
```

Illustrative flags; the team must validate them against its Docker setup. Timeouts and non-zero exits are recorded as TEST_FAILED or BLOCKED, never as success.

28. Human approval mechanism

- After the Review Agent completes, the task enters AWAITING_APPROVAL and the Orchestrator stops.
- The dashboard shows diff, test output, review findings and uncertainty; a developer chooses Approve or Reject with an optional reason.
- The decision is stored with identity and timestamp. Only a stored Approve decision allows the pull request step; Reject ends the task without PR or merge.
- The backend enforces the gate; hiding a button in the UI is not sufficient.

29. Security, privacy and permissions

Control	Implementation
Secret management	Environment variables loaded server-side; .env excluded from the repository; secrets never placed in prompts, logs or chat
Least privilege	Single-repository fine-grained GitHub token; minimal chat permissions
Isolation	Disposable container, no network, resource limits, non-root user
Prompt-injection resilience	Repository and chat text treated as data; tools constrained by allow-list; agents cannot request arbitrary commands
Privacy	Only authorised repositories and chat data processed; demo repository is synthetic
Auditability	Every agent action, tool call and decision recorded

30. Error handling and recovery

Failure	Behaviour
Malformed or invalid agent output	Bounded retry, then BLOCKED with the validation error shown
LLM timeout or quota error	Retry once; then ERROR with clear message; fixture fallback for demo mode only
Repository unavailable or permission denied	Stop safely and report
Test failure or timeout	TEST_FAILED recorded; no success claim; developer may rerun or reject
Duplicate chat event	Idempotency key prevents a second task
Incomplete bug report	NEEDS_INFORMATION; ask for details, do not invent facts

31. API design (proposed)

Method and path	Purpose
POST /api/chat/events	Receive a chat event; create task (idempotent)
GET /api/tasks	List tasks with stage and status
GET /api/tasks/{id}	Task detail: artifacts, timeline, tests, review
POST /api/tasks/{id}/approval	Record Approve or Reject decision
POST /api/tasks/{id}/retry	Retry a failed stage (bounded)
GET /api/health	Health check for demo preparation

```
POST /api/tasks/REL-001/approval
{ "decision": "approve", "decided_by": "dev@team", "reason": "Diff and tests reviewed" }

200 OK
{ "task_id": "REL-001", "current_stage": "APPROVED", "next": "PR_CREATED" }
```

Endpoint names are proposals for this project, not external API specifications.

Part C. Implementation and Validation

32. Four-member team responsibilities

Member	Owns	Deliverable
1. Backend & Architecture Lead	FastAPI, orchestrator, task states, model adapter, database, integration	Running backend, workflow controller, task API
2. AI & Agent Engineer	Triage, investigator, developer and review logic; prompts; schemas; handoffs	Working specialist agents
3. Frontend & Integration Engineer	React dashboard, Switch chat connection, GitHub integration	Usable interface and chat workflow
4. Testing, DevOps & Presentation Engineer	Docker, test harness, end-to-end QA, README, demo, slides	Repeatable demo, test evidence, documentation

Member 4 contributes to integration and test automation throughout; documentation and presentation are not an isolated full-time task.

33. MVP features

- Chat-based issue reporting and task creation (P0).
- Triage, investigation, patch proposal, testing and review agents (P0).
- Dashboard with task status, agent activity, diff and test results (P0 minimal).
- Human approval before any externally visible change (P0).
- GitHub pull request creation after approval (P1).

Definition of done. A user submits one seeded bug through the configured team channel; RELAY AI investigates, produces a patch, runs tests, shows evidence and waits for a human decision. The workflow repeats from a clean setup.

34. Prioritised feature roadmap

Priority	Feature	Decision
P0	Chat reporting, classification, investigation, fix proposal, testing, approval	Build first
P1	GitHub PR creation, task timeline, multi-task view	Add when core is stable
P2	Multiple repositories or chat platforms, voice, analytics	Not in MVP
Out	Autonomous deployment, automatic merge	Excluded

35. Development methodology

- Contract-first: agree task schema, handoff schemas and API contracts before parallel work.
- One shared repository; short-lived feature branches; one person owns the final integrated build.
- Integrate at least twice per day; keep a known-good branch.
- Every feature has an owner and a testable definition of done; freeze scope once the end-to-end demo is reliable.

36. Implementation phases

Phase	Main work	Completion check
1. Foundation	Shared repo, task format, Switch/chat connection, agent flow skeleton	A bug report reaches investigation
2. Working MVP	Patch generation, Docker tests, status view, approval	One seeded bug works end to end
3. Polish and submit	Integration fixes, UI polish, rehearsal, recorded backup demo	Three successful rehearsals; submission checked

Calendar dates are deliberately omitted: align phases with verified organiser deadlines. If time is short, cut optional features and protect the working demo.

37. Testing strategy

- **Unit:** schema validation, state transitions, approval guard.
- **Integration:** chat event to task; investigator against the demo repository; sandbox runner; GitHub adapter with a test repository.
- **End-to-end:** the seeded checkout bug from report to approval, repeated from a clean environment.
- **Failure injection:** malformed LLM output, timeout, permission denied, duplicate event.
- **Seeded bug design:** a regression test that fails before the fix and passes after it.

38. Functional and integration test cases

Test	Case	Pass condition
Functional	Valid bug report	Task created with required fields
Functional	Incomplete report	Missing details requested; nothing invented
Handoff	Investigator completes	Structured evidence reaches Developer stage
Repository	Unavailable or denied	Workflow stops safely and reports
Patch	Developer proposes change	Diff in isolated workspace; protected branch unchanged
Execution	Tests pass / fail / time out	Actual exit code and output stored; no false success
Review	Risky change	Flagged; approval still required
Approval	Human rejects	No PR or merge
Integration	Duplicate chat event	No duplicate task
Security	Malicious instruction or secret-like data in report	Tool permissions stay constrained; secrets not exposed

39. Performance and reliability metrics

Metric	Measurement	Internal MVP target
End-to-end completion	Successful seeded runs / attempts	At least 3 repeatable runs before final demo
Evidence coverage	Agent claims linked to files, logs or outputs	Every key claim evidenced or labelled hypothesis
Test reporting	Runs with captured command, exit code, output	100% of executed runs recorded
Approval enforcement	Protected actions without approval	Zero
Demo reliability	Successful rehearsals / total	At least 3 consecutive successes
Time to review-ready patch	Report to REVIEW_READY	Measured and reported, no target promised

Targets are internal team goals, not organiser thresholds or benchmark results.

40. Risks and mitigation

Risk	Impact	Mitigation
Integration failure	High	Early contracts, continuous integration, known-good branch
Inconsistent agent outputs	High	JSON schemas, validation, bounded tools, explicit transitions
Incorrect generated code	High	Tests, diff inspection, Review Agent, human approval
Unsafe command execution	Critical	Disposable container, allow-listed commands, no host shell
Switch integration takes too long	High	Minimal verified integration first; recorded fallback demo
Scope expansion	High	P0/P1/P2 list enforced
LLM quota or outage	Medium	One provider, error handling, cached demo fixtures
Demo environment failure	High	Reproducible local setup; recorded backup demo

41. Known limitations

- Narrow scope: one repository, one language, one seeded scenario.
- Model output is probabilistic; patches may be wrong or incomplete even when tests pass.
- Test coverage of the demo repository bounds how much a pass proves.
- The Switch integration depends on official capabilities not yet verified.
- No production-grade security, scalability or cost analysis has been performed.

42. Future enhancements

- Additional languages, repositories and chat platforms.
- Richer test selection and coverage analysis.
- Role-based approvals and team policies.
- Metrics dashboard from collected task data.
- Optional deployment-preview environments (with approval).

43. Deployment considerations

MVP: run locally with Docker Compose (backend, frontend, sandbox image), SQLite file, secrets in an untracked .env with a committed .env.example. Provide README steps for prerequisites, environment variables, setup, run and test, validated on a clean machine. A hosted deployment is optional and must keep the sandbox isolated. Pre-pull the sandbox image and record a backup demo.

44. Conclusion

RELAY AI proposes a focused, evidence-driven multi-agent workflow with human control at the critical boundary. The most likely reason it would underperform is incomplete end-to-end integration, not missing features. A small workflow that demonstrably works is more persuasive than a broad architecture that exists only in slides.

Appendix A. Assumptions requiring verification

- Official rules, dates, tracks, team limits, judging criteria and permitted tools for both events.
- Switch SDK/API method, supported chat providers, event model and setup instructions.
- Authorisation to connect the selected GitHub repository and chat workspace.
- Availability, quotas and terms of the chosen LLM API.
- That the demo repository is synthetic or explicitly approved for testing.

Appendix B. Example issue and resolution report (illustrative)

Field	Example
Title	Checkout total is incorrect after applying a discount
Expected	Discount applied once to the eligible subtotal; total matches the documented calculation
Actual	The discount appears to be deducted twice
Severity	Medium (illustrative demo classification)
Acceptance criteria	Regression test fails before the fix and passes after; existing checkout tests remain green

```
RELAY AI - TASK REL-001
Status:          AWAITING HUMAN APPROVAL
Issue:           Checkout total is incorrect after discount
Investigation:   Suspected duplicate discount application
Evidence:        [repository file and function reference]
Proposed change: [diff artifact / branch]
Tests:           [command executed]
Result:          [actual pass/fail count and exit code]
Review:          [findings, risks and unresolved questions]
Human decision:  Pending
No merge or deployment has been performed.
```

Bracketed fields are filled from real artifacts at run time; the report must never state a test result that was not produced by an executed command.

Appendix C. Suggested repository structure

```
relay-ai/
  backend/app/   main.py  api/  orchestrator/  sandbox/  schemas/  storage/
                 agents/        triage.py investigator.py developer.py tester.py reviewer.py
                 integrations/  switch_chat.py  github.py
  backend/tests/ requirements.txt
  frontend/src/  components/  pages/  services/
  demo-repo/     src/  tests/          (seeded checkout bug + failing regression test)
  docs/          architecture.md  workflow.md  demo-script.md
  docker/  .env.example  .gitignore  README.md
```